

SQL Performance Tuning

Best Practices & Examples

1. Use Indexes Wisely

Objective: Speed up data retrieval by reducing full table scans.

- **Create indexes** on columns frequently used in:
 - WHERE clauses
 - JOIN conditions
 - GROUP BY and ORDER BY clauses
- **Types of Indexes:**
 - **Single-column index:** Index on one column.
 - **Composite index:** Index on multiple columns. Order of columns matters.
 - **Best Practices:**
 - Avoid indexing columns with high update rates.
 - Do not over-index. It increases storage and slows write operations.

```
-- Index on WHERE column
CREATE INDEX idx_orders_status ON orders(status);

-- Composite index for JOIN
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date);
```

2. Avoid SELECT *

Objective: Reduce I/O and improve cache usage by only fetching needed columns.

```
-- Bad:
SELECT * FROM employees;

-- Good:
SELECT employee_id, first_name, last_name FROM employees;
```

3. Use Joins Smartly

Objective: Optimize data combination from multiple tables.

- Use `INNER JOIN` when matching rows only.
- Ensure join keys are indexed.
- Avoid joining unnecessary tables.

```
-- Inefficient:  
SELECT * FROM orders o LEFT JOIN customers c ON o.customer_id = c.id;  
  
-- Efficient: SELECT o.order_id, c.name  
FROM orders o JOIN customers c ON  
o.customer_id = c.id;
```

4. Filter Early (WHERE Clause Optimization)

Objective: Minimize the number of rows processed.

```
-- Inefficient:  
SELECT * FROM orders o JOIN customers c ON o.customer_id = c.id  
WHERE o.status = 'completed';  
  
-- Efficient: SELECT o.order_id, c.name  
FROM orders o JOIN customers c ON  
o.customer_id = c.id WHERE o.status =  
'completed';
```

5. Use EXISTS Instead of IN (For Subqueries)

Objective: Improve performance with correlated subqueries.

```
-- Slower:
SELECT name FROM employees
WHERE department_id IN (SELECT id FROM departments WHERE location = 'India');

-- Faster:
SELECT name FROM employees e
WHERE EXISTS (
  SELECT 1 FROM departments d
  WHERE d.id = e.department_id AND d.location = 'India'
);
```

6. Avoid Functions on Indexed Columns

Objective: Prevent index skipping.

```
-- Slower:
SELECT * FROM orders WHERE YEAR(order_date) = 2023;

-- Better:

SELECT * FROM orders
WHERE order_date BETWEEN '2023-01-01' AND '2023-12-31';
```

7. Use **LIMIT / OFFSET** for Sampling or Paging

Objective: Reduce rows returned, especially in reports or dashboards.

```
-- Return top 100 rows:  
SELECT * FROM sales ORDER BY sale_date DESC LIMIT 100;  
  
-- Pagination:  
SELECT * FROM sales ORDER BY sale_date DESC LIMIT 50 OFFSET 100;
```

8. Avoid **DISTINCT** Unless Necessary

Objective: Minimize unnecessary sorting and deduplication.

```
-- Avoid this unless needed:  
SELECT DISTINCT product_id FROM sales;
```

9. Use Aggregations Wisely

Objective: Minimize computation costs of GROUP BY and aggregates.

- Index GROUP BY columns.
- Avoid grouping on computed expressions.

```
-- Inefficient:
SELECT YEAR(order_date), COUNT(*) FROM orders GROUP BY YEAR(order_date);

-- Better:
SELECT order_year, COUNT(*)
FROM (
  SELECT order_id, EXTRACT(YEAR FROM order_date) AS order_year FROM orders
  sub
)
GROUP BY order_year;
```

10. Analyze Execution Plan

Objective: Understand how the database engine executes your query.

- Use EXPLAIN, EXPLAIN ANALYZE, or similar depending on RDBMS:
- PostgreSQL: EXPLAIN ANALYZE
- MySQL: EXPLAIN
- SQL Server: Actual Execution Plan (GUI)

Example:

```
EXPLAIN SELECT * FROM orders WHERE status = 'shipped';
```

Bonus Tips:

- Partition large tables.
- Use proper data types.
- Use query result caching if available (e.g., Snowflake, BigQuery).
- Rewrite complex queries with CTEs or temp tables.

Would you like this exported to a PDF or customized for a specific SQL dialect (e.g., Snowflake, PostgreSQL, SQL Server)?

Thank you

If You Found This
Helpful?

